

Lesson 6

At this point, we are not going to worry about the line numbers being exactly correct in the text and figures. These will vary as you write your program, so look at the text to find your position.

You've already learned a lot about programming in the first five lessons, but there's also been a hand-waving of concepts by saying "ignore this for now, we'll learn about it later". This has enabled you to spend more time focusing on writing code and playing with the robot. However, it can also feel a little overwhelming when you're looking at a lot of code that you don't fully understand. Similar to what we did in lesson four with methods, this lesson is going to focus on helping you build a deeper understanding of object-oriented programming and what classes and objects are. These are some of the fundamentally most important concepts in Java and many other programming languages, so take your time with this lesson, focus, repeat the reading and/or videos as necessary, and don't worry if it seems overwhelming at first. If you go through the material and projects carefully, you *will* come to understand it, and that will help you understand what's going on in the robot code much better as well. For the hands-on portion of this project, first you'll follow along with a video, and then you'll modify the Romi code to use two of the built-in LEDs – red and green – to indicate whether or not the Romi has been rotated from its original orientation (for example, if you were to drive it and it started to veer off course instead of driving straight ahead.)

Start out by watching this video and following along in a blank non-robot project in VSCode. If you don't remember how to set up a non-robot project in VSCode, you can refer back to lesson 4. This video gives a simple explanation of objects, how they have methods and variables, and how you can use them: [OOP video](#)

After you watch the video and code along with it, read this page, from the start through section 2.9. Don't worry about anything that comes after section 2.9:

[OOP Basics](#)

Now that you've made your way through these materials, you should have a decent idea of what object-oriented programming (OOP) is, and robot code will probably make a little more sense to you. There are still a lot of concepts to cover in part two, but you've done enough reading for one lesson so let's get our hands dirty with some Romi code.

Measuring Orientation: Gyroscopes

The Romi has a built-in gyroscope, or *gyro*, which is a device that measures changes in orientation of the robot and can also add up all the changes over time to measure the current orientation at any given moment. Having access to this data opens up a lot of possibilities when it comes to robot code. One example of this is the autonomous program you saw run in the very first lesson, where the Romi turns around twice. How does it know how much to turn, to turn around without turning too far (or not far enough)? While this could be achieved by measuring exactly how far each wheel spins, the most reliable way to do this is to use the gyro, because, for example, the wheel could slip. Another great use of a gyro is detecting whether or not your robot is driving straight, because there are a lot of ways your robot

could drive a bit crooked. One motor might be slightly more powerful than the other. One side of the drivetrain might have more mechanical resistance than the other. The surface you're driving on might be a little bit uneven. Those are just a few examples; there are a lot of things that can go wrong, but with a gyro, you can detect all these things, and correct for them. In a future lesson we'll correct for today, we'll just worry about detecting them. We'll write a program that uses the built-in lights to light up either red or green, based on whether or not the Romi is still facing the same direction it was when the program started running.

Accessing Class Data

To do this project, we're going to need to access a few variables that, by default, we can't access. We'll talk more about this in the second part of the object-oriented programming deep dive, next lesson. There are reasons why these variables should not be accessible to us, but for this project, we're going to make them accessible anyway. Follow these steps to get everything setup for this project:

1. Create a new RomiReference project like you've done a couple times already.
2. In RobotContainer.java on line 31, there are two instances of the word "INPUT". Change these both to "OUTPUT". This tells the Romi to use the red and green LED lights as output.
3. In Drivetrain.java on line 34, change the first word of the line, "private", to "public".

With that out of the way, it's time to get started. This program will utilize your newly gained knowledge about classes by accessing variables belonging to various objects in the program. In this project, we're going to create a command that uses two subsystems: the drivetrain, which is where the gyro exists, and the on-board IO, which is where the lights exist.

You're now ready to create the command that will actually *do* things. Create a new command in the commands folder called ToggleLightsBasedOnGyro.

```
public class ToggleLightsBasedOnGyro extends Command {
    private final Drivetrain m_drivetrain;
    private final OnBoardIO m_io;

    /** Creates a new ToggleLightsBasedOnGyro. */
    public ToggleLightsBasedOnGyro(Drivetrain drivetrain, OnBoardIO onboardIO) {
        m_drivetrain = drivetrain;
        m_io = onboardIO;
    }
}
```

You may get an error because you may not have imported Drivetrain and OnBoardIO.

Now it's time to add the logic that reads the angle from the gyro and then turns the lights on and off.

From your ToggleLightsBasedOnGyro class, you can use both of these classes. The RomiGyro class has a method called “getAngleZ()”, which returns a double value that represents an angle in degrees (you can see this if you open RomiGyro.java in the “sensors” folder, and look on line 108.) As you learned in the materials for this lesson, you can use dots (periods) to chain through your objects. So to call the getAngleZ() method, you can use your class variable m_io, and do “m_io.m_drivetrain.m_gyro.getAngleZ()”.

While a command is running, its *execute* method is called continuously. We want our lights to update continuously, so this is where we’ll put our code. Navigate to the execute() method in your command (approximately line 24), and as the first line of the method, declare a variable and set it equal to the output from the getAngleZ() method call described above. Here’s an example:

```
public void execute() {  
    // Get the current angle from the gyro  
    double gyroAngle = m_drivetrain.m_gyro.getAngle();  
}
```

Now you have a variable that contains the current orientation of the robot, relative to the robot’s orientation when the program started running, which will be whenever you press F5 and the code loads onto the robot. Since execute() is called continuously while the command is running, this variable will continue to be updated during execution.

Now that you have the angle, it’s time to turn lights on and off. To do this, you’ll want to use two methods belonging to the OnBoardIO class – setGreenLed(boolean) and setRedLed(boolean). Notice how those methods are written here in plaintext, with the word “boolean” in the parentheses. This demonstrates that in order to use them, you need to send a boolean variable. This makes sense – lights have two states, on or off, so you can pass either true or false to the methods to turn the light on or off. There is one method for the green light and one method for the red light. In this project, we’ll set the green light to on if we’re on course, and the red light to on if we’re off course. In both cases we’ll set the other light to off, so there will only ever be one light on at a time.

You might want to turn the lights to one configuration, and you might want to turn them to a different configuration, depending on the gyro angle. That means you’re going to need a conditional statement. “Conditional” is just a fancy word for an if statement. If the gyro angle is zero, then the robot is facing straight ahead, and if it’s not, then the robot has turned a little bit. Of course, in practice, the gyro is sensitive, and the angle is not going to be *exactly* zero, so you’re going to need to check if it’s in some range. If you make the range too big, say, 90 degrees, then you could veer far off course before detecting it. If you make the range too small, say, one quarter degree, then you’re going to detect that you’re off course even when you’re mostly straight. Finding the perfect range depends on a lot of things, but for this project, a range of +/- five degrees works pretty well.

You could measure if the gyro angle is less than 5 degrees pretty easily, with a line like this:

```
if (gyroAngle < 5) {
```

However, the robot can turn in two different directions. If you turn the robot to the right, the gyro angle will increase. If you turn to the left, it will *decrease*. This means that if you turn 90 degrees to the left relative to the starting point, the gyro angle would return -90. This is indeed less than 5 degrees, so the

if statement above would return true, even though you'd be very far from straight. Thankfully, there's an easy solution to this, using something called "absolute value". The absolute value of a number is simply that number, but *not* negative. For example, the absolute value of 10 is 10. The absolute value of -10 is also 10. So the only difference is that if the number is negative, it becomes positive. If it's already positive, there's no change. Java has a built-in method to return the absolute value of any number: `Math.abs(double)`. `Math.abs(gyroAngle)` will return whatever the gyro angle is, but positive. You can use this function in your if statement to check if the absolute value of the gyro angle is less than five. Write this if statement, and if it's true, set the green LED to true and the red one to false, and vice versa if the if statement is false. Here's an example:

```
// Called every time the scheduler runs while the command is scheduled.
@Override
public void execute() {
    // Get the current angle from the gyro
    double gyroAngle = m_drivetrain.m_gyro.getAngle();

    // If the angle is greater than 10 degrees, turn on the lights
    if (Math.abs(gyroAngle) < 5) {
        m_io.setGreenLed(value: true);
        m_io.setRedLed(value: false);
    } else {
        m_io.setGreenLed(value: false);
        m_io.setRedLed(value: true);
    }
}
```

We could be done with this command at this point, but let's make a couple quick upgrades before we finish. Since we're still practicing our mastery of using methods, let's split out lines 28 and 29, and 32 and 33 above, into their own methods. Beneath the `execute()` method, create two more methods called "`setLightsGreen()`" and "`setLightsRed()`". You can declare both of these with "`private void`". Move the lines of code identified above into these methods, and inside your if statement in the `execute` method, call your newly created methods. Lastly, you may want to add a SOP line that outputs the gyro angle, so you can see it in real time in the terminal. This isn't strictly necessary because you can also see it on the robot simulation interface, but you can add it if you want. Here's an example of the command with all these steps completed:

```

24 public void execute() {
25     double gyroAngle = m_io.m_drivetrain.m_gyro.getAngleZ();
26
27     if (Math.abs(gyroAngle) < 5) {
28         setLightsGreen();
29     }
30     else {
31         setLightsRed();
32     }
33
34     System.out.println("Gyro angle: " + gyroAngle);
35 }
36
37 private void setLightsGreen() {
38     m_io.setGreenLed(true);
39     m_io.setRedLed(false);
40 }
41
42 private void setLightsRed() {
43     m_io.setGreenLed(false);
44     m_io.setRedLed(true);
45 }

```

With that done, your command is completed. Now there's just one tiny step to go. Navigate to the end of line 54 in RobotContainer.java, press enter, and then add four lines like so:

```

52 public RobotContainer() {
53     // Configure the button bindings
54     configureButtonBindings();
55     m_onboardIO.setGreenLed(true);
56     m_onboardIO.setGreenLed(false);
57     m_onboardIO.setRedLed(true);
58     m_onboardIO.setRedLed(false);
59 }

```

All this does is initialize the values so that the lights turn on as soon as your code starts. With that, you're done – deploy your code! Make sure not to rotate your robot by hand before you enable it. When you enable teleoperated mode, you'll see the green light illuminate. If you start rotating your robot, either by driving it or by hand, you'll see the lights change as the angle of the robot moves away from its initial orientation and change back if you move the robot back.

The only thing left to do is to start your command. In RobotContainer, go to about line 64 and add the trigger as in this figure:

```

private void configureButtonBindings() {
    // Default command is arcade drive. This will run unless another command
    // is scheduled over it.
    m_drivetrain.setDefaultCommand(getArcadeDriveCommand());

    Joystick joystick = new Joystick(port: 0);

    new JoystickButton(joystick, buttonNumber: 2)
        .toggleOnTrue(new ToggleLightsBasedOnGyro(m_drivetrain, m_onboardIO));
}

```

If the Gyro has too much drift, you can calibrate it by opening your browser with wifi connected to your Romi. Go to network address 10.0.0.2 and follow this wpilib procedure. [Gyro Calibration](#) Go down the page to the IMU section. Make sure you are in the writable section and after calibration, go back to Read-Only.

In this lesson, you learned about OOP, and saw an example of how you can use classes and objects in code to logically interact with different parts of your program through a class hierarchy. You also saw an example of how you can integrate different subsystems on the Romi with each other to create some cool functionality. This was a longer lesson with a lot of important concepts, so don't worry if you feel like you need to review any part of it. Although there are still a couple deep dives left to go before you'll understand everything you see in the code, you're making great progress. We'll continue to build on these skills in future lessons.